

Pontifícia Universidade Católica do Paraná

**Relatório — PBL 03: Implementação de Tabelas Hash
com Diferentes Funções Hash**

Resolução de Problemas Estruturados em Computação

Bacharelado em Engenharia de Software

Aluno: ALANDER MENEZES ARANTES DE ÁVILA

1. Introdução

A tabela hash, também conhecida como tabela de dispersão ou espalhamento, é uma estrutura de dados que associa chaves a valores utilizando uma função hash determinística e deliberadamente projetada para determinar a posição de armazenamento dos dados na memória.

No presente relatório serão apresentados os resultados da implementação de duas tabelas hash em Java, utilizando funções hash distintas e o método de **endereçamento aberto homogêneo com sondagem linear** para tratamento de colisões. O objetivo é comparar a eficiência de cada abordagem em termos de número de colisões, tempo de inserção, tempo de busca e distribuição das chaves na tabela.

Os dados utilizados consistem em uma lista de 20.000 nomes reais distintos, carregados a partir de um arquivo CSV. A tabela é inicializada com capacidade 16 e redimensionada automaticamente sempre que o fator de carga de 0,75 é atingido.

2. Implementação

2.1 Arquitetura do Projeto

O projeto foi desenvolvido em Java utilizando Programação Orientada a Objetos. A estrutura de classes segue o modelo de uma classe abstrata genérica e duas subclasses que sobrescrevem apenas a função hash.

Classe	Responsabilidade
TabelaHash (abstrata)	Define estrutura, inserção, busca, remoção e redimensionamento
TabelaHashSoma	Implementa hash por soma dos valores ASCII dos caracteres
TabelaHashDJB2	Implementa hash DJB2 (multiplicação por 33 a cada caractere)
LeitorCSV	Lê o arquivo CSV e retorna lista de nomes
TesteEficiencia	Mede tempos de inserção e busca, gera relatório
Main	Ponto de entrada completo

2.2 Tratamento de Colisões — Endereçamento Aberto Homogêneo

O método escolhido foi o endereçamento aberto homogêneo com sondagem linear. Quando uma colisão ocorre durante a inserção, o algoritmo percorre linearmente as posições seguintes até encontrar uma vaga disponível. Na busca, o processo é idêntico: partindo da posição calculada pela função hash, percorre-se linearmente até encontrar a chave ou uma posição nula (indicando ausência do elemento).

Para evitar o problema de falso negativo na busca após remoções, posições removidas recebem o marcador especial "**__REMOVIDO__**", diferenciando-as de posições nunca ocupadas.ent

2.3 Redimensionamento

A tabela é redimensionada quando o número de elementos atinge o limite de inserção, calculado como: **capacidade × fator de carga (0,75)**. Ao redimensionar, a capacidade é dobrada e todos os elementos são reinseridos via *rehashing* para as novas posições.

3. Funções Hash Implementadas

3.1 Hash Soma de Char

Esta função soma os valores ASCII de todos os caracteres da string e aplica o operador módulo pelo tamanho da tabela. É uma função simples e de baixa complexidade computacional, porém com distribuição fraca para nomes em português, pois nomes com os mesmos caracteres (em qualquer ordem) produzem o mesmo hash.

```
Fórmula: hash = (Σ char[i]) % capacidade
```

3.2 Hash DJB2

A função DJB2, criada por Daniel J. Bernstein, utiliza uma multiplicação acumulada por 33 a cada caractere, iniciando com o valor semente 5381. O deslocamento de bits ($hash \ll 5$) + $hash$ é equivalente a multiplicar por 33, sendo eficiente e produzindo boa dispersão. É amplamente utilizada em compiladores e sistemas de hash por sua excelente distribuição.

```
Fórmula: hash = 5381; hash = (hash * 33) + char[i]; retorna hash % capacidade
```

4. Resultados dos Testes de Eficiência

4.1 Comparativo Geral

Métrica	Hash Soma de Char	Hash DJB2
Total de colisões	272.167.038	53.789
Tempo de inserção	1.538,808 ms	12,135 ms
Tempo de busca	1.655,661 ms	4,706 ms
Capacidade final	32.768	32.768
Elementos inseridos	20.000	20.000

4.2 Análise das Colisões

A diferença no número de colisões é expressiva: a Hash Soma gerou aproximadamente **5.060 vezes mais colisões** que a DJB2. Isso ocorre porque nomes em português compartilham muitos dos mesmos caracteres e comprimentos similares, fazendo com que a simples soma dos ASCII produza valores muito concentrados em uma faixa estreita da tabela. A clusterização severa é visível na distribuição: praticamente todas as 20.000 chaves ficaram comprimidas entre as posições 1.100 e 21.300 da tabela de 32.768 posições, deixando 34% do espaço completamente vazio.

A DJB2, por sua vez, distribuiu os elementos de forma uniforme por toda a tabela, com variação entre 39 e 83 chaves por bloco de 100 posições — resultado próximo ao ideal teórico de ~61 chaves por bloco (20.000 / 328 blocos).

4.3 Análise dos Tempos

O impacto das colisões no desempenho é direto. A Hash Soma levou **1.538 ms para inserir** os 20.000 nomes, contra apenas **12 ms da DJB2** — uma diferença de mais de 126 vezes. Na busca, a diferença foi de 1.655 ms contra 4,7 ms, aproximadamente **352 vezes mais lento**. Isso demonstra o custo da sondagem linear em cenários de alta clusterização: ao tentar inserir ou buscar um elemento, o algoritmo precisa percorrer longas cadeias de posições ocupadas até encontrar o destino correto.

4.4 Distribuição das Chaves

A distribuição reforça a análise acima. A tabela abaixo resume a ocupação por região:

Região da tabela	Hash Soma de Char	Hash DJB2
Posições 0 – 1.099	0 chaves (0%)	Ocupação uniforme
Posições 1.100 – 21.300	~20.000 chaves (100%)	Ocupação uniforme
Posições 21.301 – 32.767	0 chaves (0%)	Ocupação uniforme
Variação por bloco de 100	de 0 a 100	de 39 a 83

5. Conclusão

Os resultados demonstram de forma clara a importância da escolha da função hash. A Hash Soma de Char, apesar de simples, é inadequada para chaves do tipo string em português, pois ignora a ordem dos caracteres e produz valores muito concentrados. Isso resulta em clusterização severa e degrada o desempenho da tabela hash para próximo de uma busca sequencial no pior caso.

A DJB2 apresentou desempenho superior em todos os critérios avaliados: colisões ~5.000 vezes menores, inserção ~126 vezes mais rápida e busca ~352 vezes mais rápida. Sua distribuição uniforme valida os princípios de uma boa função hash: baixo número de colisões, baixa complexidade computacional e uniformidade na distribuição das chaves.

O trabalho também evidenciou o funcionamento correto do redimensionamento com fator de carga 0,75 e do endereçamento aberto homogêneo com sondagem linear, ambos funcionando de forma idêntica nas duas tabelas, isolando a função hash como única variável de comparação.